Parfait Loïc — tu veux comprendre **tout le cheminement complet** quand tu cliques sur :

- ✅ le bouton ✅ (marquer terminé / non terminé)
- ❌ le bouton ❌ (supprimer)

Je vais te montrer **exactement ce qui se passe**, étape par étape, **du frontend → backend → base de données → retour frontend**.

Et je vais te commenter **chaque ligne** pour que tu comprennes parfaitement.

🟩 1. Quand tu cliques sur ✅ (toggleDone)

🎯 Étape 1 — FRONTEND : l'utilisateur clique

Dans `index.html` :

```
<button @click="toggleDone(task)" class="btn-small">✓</button>
```

➡️ Vue appelle la méthode `toggleDone(task)` dans `app.js`.

🎯 Étape 2 — FRONTEND : méthode toggleDone()

```
async toggleDone(task) {

  // On crée une copie de la tâche avec done inversé
  const updated = {
    ...task,
    done: task.done == 1 ? 0 : 1
  };

  // On envoie la requête PUT à l'API
  await fetch(`${this.apiBaseUrl}/${task.id}`, {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(updated)
  });

  // Mise à jour locale de l'interface
  task.done = updated.done;
}
```

🔍 Ce qui se passe ici :

1. Vue reçoit la tâche cliquée
2. Vue inverse `done` (0 → 1 ou 1 → 0)
3. Vue envoie une requête HTTP :

```
PUT http://api.localhost/tasks/3
```

Avec ce JSON :

```
{
  "id": 3,
  "title": "Acheter du pain",
  "done": 1
}
```

4. Vue met à jour l'interface **sans attendre la réponse** (optimisation UX)

🎯 Étape 3 — BACKEND : index.php reçoit la requête

```
if ($method === "PUT" && isset($uri[1])) {
    $controller->update($uri[1]);
}
```

➡️ L'API appelle :

```
TaskController->update(3)
```

🎯 Étape 4 — BACKEND : TaskController update()

```
public function update($id) {
    $data = json_decode(file_get_contents("php://input"), true);

    $task = new Task();
    $task->update($id, $data);

    Response::json(["message" => "updated"]);
}
```

🔍 Ce qui se passe :

1. Le backend lit le JSON envoyé par Vue
2. Il appelle le modèle Task
3. Il met à jour la base de données
4. Il renvoie :

```
{ "message": "updated" }
```

🎯 Étape 5 — BACKEND : Task.php met à jour la DB

```
public function update($id, $data) {
    $stmt = $this->db->prepare("UPDATE tasks SET title=?, done=? WHERE id=?");
    $stmt->execute([$data['title'], $data['done'], $id]);
}
```

➡️ La tâche est mise à jour dans MySQL.

🎯 Étape 6 — FRONTEND : Vue met à jour l'affichage

```
task.done = updated.done;
```

➡️ La classe CSS `.done` s'active ou se désactive :

```
.task-item.done span {
  text-decoration: line-through;
  opacity: 0.6;
}
```

🟥 2. Quand tu cliques sur ❌ (supprimer)

🎯 Étape 1 — FRONTEND : clic utilisateur

Dans `index.html` :

```
<button @click="deleteTask(task.id)" class="btn-small btn-danger">✕</button>
```

➡️ Vue appelle `deleteTask(id)`.

🎯 Étape 2 — FRONTEND : méthode deleteTask()

```
async deleteTask(id) {

  // Requête DELETE vers l'API
  await fetch(`${this.apiBaseUrl}/${id}`, {
    method: 'DELETE'
  });

  // Mise à jour locale : on retire la tâche du tableau
  this.tasks = this.tasks.filter(t => t.id !== id);
}
```

🔍 Ce qui se passe :

1. Vue envoie :

```
DELETE http://api.localhost/tasks/3
```

2. Vue supprime la tâche de l'interface
3. Pas besoin d'attendre la réponse pour afficher le changement

---

🎯 Étape 3 — BACKEND : index.php route la requête

```
if ($method === "DELETE" && isset($uri[1])) {
    $controller->delete($uri[1]);
}
```

➡️ Appelle :

```
TaskController->delete(3)
```

🎯 Étape 4 — BACKEND : TaskController delete()

```
public function delete($id) {
    $task = new Task();
    $task->delete($id);

    Response::json(["message" => "deleted"]);
}
```

🎯 Étape 5 — BACKEND : Task.php supprime la tâche

```
public function delete($id) {
    $stmt = $this->db->prepare("DELETE FROM tasks WHERE id=?");
    $stmt->execute([$id]);
}
```

➡️ La tâche est supprimée de MySQL.

🎯 Étape 6 — FRONTEND : Vue met à jour l'affichage

```
this.tasks = this.tasks.filter(t => t.id !== id);
```

➡️ La tâche disparaît immédiatement de l'interface.

🧭 SCHÉMA VISUEL DU FLUX (toggleDone)

```
[Utilisateur clique ✓]
        ↓
[Vue → toggleDone()]
        ↓
[PUT /tasks/3]
        ↓
[index.php → route PUT]
        ↓
[TaskController->update()]
        ↓
[Task->update() → MySQL]
        ↓
[Réponse JSON]
        ↓
[Vue met à jour task.done]
```

```
        ↓
[CSS applique .done]
```

🧭 SCHÉMA VISUEL DU FLUX (deleteTask)

```
 [Utilisateur clique ✕]
        ↓
[Vue → deleteTask()]
        ↓
[DELETE /tasks/3]
        ↓
[index.php → route DELETE]
        ↓
[TaskController->delete()]
        ↓
[Task->delete() → MySQL]
        ↓
[Réponse JSON]
        ↓
[Vue retire la tâche du tableau]
```

🎉 Résultat

Tu comprends maintenant **tout le cheminement complet**, du clic utilisateur jusqu'à la base de données, et le retour vers l'interface.

Si tu veux, je peux aussi te faire :

- un **schéma graphique PNG** (architecture complète)
- une **documentation technique** de ton API
- un **diagramme MVC**
- une **explication du routage Apache + .htaccess**

Tu veux lequel ?